Software Engineering in the Age of AI

Arsham Gholamzadeh Khoee

1 Introduction

In recent years, foundation models and large language models (LLMs) have become an important component of modern AI systems that can perform a wide range of tasks because of their language understanding, reasoning, and code generation. These models are highly versatile and can be used as a core building block of solutions that deal with real-world applications. However, in high-stakes domains such as automotive software engineering, producing an answer that only sounds plausible is not sufficient. Such systems have to consistently follow domain rules, operate within fixed data structures and operational constraints. And more importantly, the reasons and answers provided by such a system must be verifiable by a domain expert. To achieve this, we need to address the gap between the general capability of foundation models and the domain-specific reasoning in safety-critical applications.

As a result, my research focuses on filling this gap by designing AI systems that are (i) generalizable across domains, (ii) adaptable to a target domain, and (iii) able to reason reliably within a specific domain's rules. These three levels build on each other [1]. Generalization gives the model a robust foundation that transfers across unseen domains. Adaptation aligns this foundation with the specifics of a target domain. Domain-specific system design then enforces the rules, workflows, and constraints that reliable use in practice requires.

My current work builds on the ideas outlined above by applying them to LLM-based systems solving real-world problems. One focus is harness engineering: designing the tools, control flow and execution environments (or "harnesses") with which an agent will operate in order to ensure it does not generate freely but remains focused on the task at hand. A second focus is improving reasoning, structuring how a model plans, checks intermediate steps, and corrects its own mistakes during multi-step problem solving. A third focus is memory and prompt optimization: deciding what context an agent keeps across steps and how that context is selected and summarized, so each call receives what the task needs rather than everything available. These topics have been applied to various types of systems, including those used for code generation, data analysis and multi-step task automation, in order to turn general LLM capabilities into reliable working systems capable of dealing with the challenges faced by real tasks beyond curated benchmark applications.

2 Lecture Principles

2.1 Technical Debt in AI-Based Systems

From a software engineering perspective, harness engineering for LLM-based automation can introduce its own technical debt, sometimes outweighing the debt it was built to remove. Companies increasingly build agent harnesses, the tools, control flow, and execution scaffolding surrounding a model, to automate repetitive engineering tasks. Automated testing is a representative case: LLMs have sharply increased the speed of software generation, but the generated code still needs to be tested to confirm it behaves as intended and does not silently accumulate defects and show up later. A working automated testing pipeline is therefore a genuine opportunity for LLM-based automation to close a real gap in the software development process.

However, in practice, testing complex and interactive systems can be significantly more difficult than testing isolated functions. Mobile UI testing illustrates this well: an end-to-end, intent-driven testing agent [3] must explore an application's screens, choose the correct action at each step, tapping, swiping, or entering text, and interpret the resulting visual state accurately enough to know whether that action succeeded. Companies have built agentic frameworks for exactly this kind of task, but these frameworks tend to perform well

mainly on curated benchmarks; once deployed on real applications outside that setting, they must handle far more varied states and action sequences with the same precision, which existing harnesses often cannot guarantee.

The harnesses built to solve this problem can themselves become a source of technical debt, for two related reasons. First, LLMs improve quickly, so a harness tied to a specific model needs that model swapped out repeatedly to stay competitive, turning what should be a one-time integration into an ongoing maintenance cycle that has to keep pace with the release schedule of the models it depends on. Second, current models are trained and optimized largely for coding tasks within specific, well-tuned harnesses, such as Claude Code, rather than for other tasks, such as testing, or for other custom-built harnesses; dropping the same model into a different, customized harness can introduce compatibility issues that only surface once the harness is embedded in a larger system.

The result is an engineering trade-off worth stating explicitly. A custom harness built in-house to reduce technical debt in one part of the system, such as testing, can add technical debt elsewhere: in the harness itself. The cost shows up as the recurring work of tracking model updates and resolving harness-model incompatibilities. Fine-tuning a model to fit a specific harness or task is one way to work around this mismatch. But it does not remove the underlying dependency, as each new base-model release would call for re-tuning to stay current, so the recurring cost just shifts from harness maintenance to fine-tuning maintenance. For the same reason, in some cases the more economical choice is neither to build a customized model nor to fine-tune a model for it, but to rely on general-purpose harnesses maintained by model providers, for example Claude Code combined with Model Context Protocol (MCP) tools, in order to avoid a single team from having to bear the recurring cost of tracking model updates. For such tasks, this points to a simple rule: default to the general-purpose harness, and justify a custom-built alternative only when it demonstrates a level of accuracy and reliability that the general-purpose option cannot achieve.

2.2 Metamorphic Testing in the Era of Agentic Systems

Text-to-SQL systems illustrate a correctness requirement that is easy to state but hard to test: if a user asks the same question twice, once in one phrasing and once in another, the two generated SQL queries should produce the same result when executed against the database, since SQL execution itself is deterministic. LLMs have made this kind of translation from natural language to structured queries much easier, and the resulting systems now support real applications that range from analyzing server logs in data centers, to interpreting tabular test data for software release decisions [4], to giving sales managers accurate statistics against their KPIs. In all of these settings, the natural-language interface is only useful if it stays correct as users phrase the same request in different ways.

Measuring this kind of correctness on curated, open-source text-to-SQL benchmarks is not sufficient. These benchmarks pair each natural-language question with a single fixed query and result, and because their questions, schemas, and queries are publicly available, an LLM's strong benchmark performance may partly reflect exposure to these exact patterns during pretraining rather than a generalizable ability to translate arbitrary phrasings of an input query into correct SQL.

One of the main problems arises when such a system leaves the benchmark and meets real users. These models are stochastic. The same model can reason through different requests. Even a small change in wording, or a different dialect of the same request, can shift the model's reasoning trajectory and lead it to generate a different query, one that may no longer be correct. Two users asking the same question of the system may expect a single answer, but there may be no guarantees in the model that a slight change in how a request is phrased will preserve the reasoning that was needed to formulate the best possible question. Benchmarks that are built with a single fixed phrasing cannot identify this issue, since they never exercise the same intent in different ways.

Metamorphic Testing (MT) [2] is incredibly helpful for the gap described here. Traditional testing in-

volves checking a system for functionality based on input/output pairs. MT on the other hand checks how a natural language question is phrased and how the system acts accordingly. For user interfaces that communicate via natural language, MT is a great way to check how robust a system is. This is especially useful for systems that require natural language input over natural language output. Metamorphic testing directly relates to how agents guide their behavior and makes semantic drifting more measurable and less reliant on isolated anecdotal examples.

Determining the correct metamorphic relation is just as important as running the tests. In this example, the appropriate metamorphic relation is that semantically equivalent natural language inputs correlate with equivalent SQL query execution results, not equivalent textually defined SQL queries. Two SQL queries that differ in join order or the formulation of a subquery can be also considered correct. Therefore, requiring generated text to match could mean a successful execution is a failure. Testing at the execution result level and not at the query text level shows how metamorphic testing is a proper means to assess correctness rather than the particular style of query generation.

3 Guest-Lecture Principles

3.1 AI for Software Engineering and Software Engineering for AI

Software engineering is an iterative process built from several activities including planning, risk analysis, requirement engineering, design, coding, versioning, testing, integration, deployment, and maintenance, and the software that results from this process typically exists to automate or support some procedure. The arrival of LLMs changes this picture in two directions at once: AI can now be used to carry out parts of this process, and the systems produced this way, in turn, have to be engineered with the same rigor as any other piece of software.

In the first direction, AI for software engineering, LLMs let parts of the engineering process itself be delegated rather than carried out entirely by hand. Coding, and increasingly requirement engineering, no longer have to be performed by a person at every step: an LLM can draft an implementation from a specification or turn a stakeholder's request into a structured requirement, tasks that previously needed a human because they depend on interpreting intent expressed in natural language rather than following a fixed procedure. The same holds true for both the software engineering process and the software itself. It is now possible to invoke an LLM instead of hand-coding logic for functions that resisted direct implementation as algorithms, such as understanding a natural-language query, planning over a long series of steps, or deciding what action to take given the current state of a task. In this way, a system can now process raw data directly and respond to users directly, without requiring large amounts of protocol or personnel.

The second direction, software engineering for AI, is the reverse problem: once an LLM is embedded as a component inside a larger system, that system, its harness, prompts, memory, and the reasoning procedure wrapped around the model, has to be planned, designed, versioned, tested, and maintained with the same discipline as any other software component. Treating a model as "just a function call" understates the engineering effort this actually requires. The technical debt that a custom-built harness can accumulate, and the need for metamorphic testing to check that a natural-language interface behaves consistently under rephrasing, discussed as Topics 1 and 2 above, are both concrete instances of this direction: they are software engineering problems that only exist because an LLM, not a deterministic program, sits inside the system.

These two directions feed each other: as AI takes on more of the software engineering lifecycle, the systems doing that work become software in their own right, and their correctness cannot be assumed just because a capable model sits inside them. My research sits at this intersection: using LLMs to automate parts of the engineering process, and engineering the resulting AI-based systems with the same rigor as any other software.

3.2 The Future of Software Engineers in the Era of LLMs

With the development of LLMs, code can be generated directly from natural language descriptions, thereby eliminating the need to write code by hand. Consequently, defining precisely what is needed, defining the architecture that the generated code must fit into, and reviewing and verifying that the components function as intended have become paramount. The engineering work shifts from producing code to defining and checking it.

This shift is somewhat a consequence of one of the limitations of current LLMs: their output quality depends heavily on how a task is specified, on the wording, structure, and detail of the input prompt, and on how consistently a user steers the model toward the intended solution. Two lines of work are addressing this from different directions. Prompt optimization techniques, such as GEPA [8], search for prompts that reliably elicit the intended behavior from a fixed model. Meanwhile, model providers are improving the internal thinking and reasoning procedures of these systems, aiming to make their outputs less sensitive to how a request happens to be phrased. Nevertheless, until these efforts close the gap, the quality of software produced with an LLM depends substantially on the knowledge and experience of the engineer directing it, since prompting a model effectively still requires understanding the problem well enough to specify it precisely.

Despite this dependency, deploying LLMs within software engineering teams already increases the speed of code production enough to reduce the number of developers needed to produce a given amount of working code. This changes what teams need people for. If code generation itself becomes faster and requires less direct involvement, the effort that determines whether the system is reliable moves to code review, testing, and verification. Roles that were previously secondary to writing new code, such as infrastructure engineering and quality assurance, take on a larger share of responsibility for whether a system works, since these are the layers that capture what generation alone is unable to guarantee.

Several aspects of my own research relate to this shift: developing prompt optimization techniques that make LLM behavior less dependent on how a request happens to be phrased, building systems that incorporate domain-specific knowledge so that generated outputs respect the constraints of the target domain rather than only the constraints implicit in the prompt, and decomposing high-level requirements into steps that a model, or a pipeline of models, can execute reliably.

4 Data Scientists, Software Engineers, and AI Engineers

Machine Learning in Production [5] introduces data scientists as people with an educational background in statistics and machine-learning algorithms, focused on the data and the model: they spend much of their time gathering and cleaning data, engineering features, and training models to reach good accuracy on held-out test data. In contrast, software engineers are introduced with a background in the traditional computer science and engineering disciplines, requirements engineering, software design, distributed systems, and quality assurance, focused on system architecture, budgets, and delivery within a given time; rather than being judged on accuracy, their work is evaluated against trade-offs among qualities such as usability, scalability, maintainability, security, development time, and cost. The book identifies both of these as distinct disciplines with distinct knowledge and responsibilities. The authors note that strong people in both disciplines are very rare and are called "unicorns." They state that it is simply not possible for one discipline to build a production ML system. They believe teams should consist of "T-shaped" members where a person has deep knowledge in one discipline and shallower knowledge in the other to have effective communication across the boundary. The authors of the book identify a more recent discipline, the "AI engineer", where the focus tends to be automating pipelines, deploying, and MLOps, and is slightly more model-centered than system-centered.

I agree with this picture, but I think the boundary it describes is shifting with the emergence of LLMs. As machine learning has moved from training custom models to building systems around general-purpose mod-

els accessed through prompts and APIs, much of the work that used to separate data scientists and software engineers, feature engineering, model architecture, hyperparameter tuning, no longer needs to happen at all for many applications. What replaces it, designing the harness, prompts, and control flow around a model, requires the same system-level thinking regardless of which side of the traditional split someone came from. Data scientists and software engineers use similar reasoning for the same concepts such as cost, prompting, and model evaluation. A clear picture of how data scientists and software engineers differ is becoming blurrier in the context of LLMs. LLMs also compensate for skills either role tends to lack on its own, such as helping a software engineer reason about model behavior or helping a data scientist write and maintain production-quality code, which further narrows the practical skill gap between the two roles.

I would push the book's characterization of the AI engineer further.I see AI engineering as a role that combines software engineering, systems architecture, and data science to bridge machine-learning concepts and scalable software products. The book's critique, that it still skews model-centric rather than system-wide, is a fair description of where the role currently stands. But it is not where the role is heading. As I argue below, the bottleneck in deploying LLM-based systems is increasingly infrastructure, not modeling. I expect the AI engineer role to converge toward owning that infrastructure directly: deploying models and engineering the environment and infrastructure around them. Concretely, this means fluency with cloud computing platforms and accelerated computing, for example managing NVIDIA GPUs, orchestrating workloads with Kubernetes, and working with distributed training and inference, skills that sit closer to systems engineering than to the data-science end of the spectrum the book currently associates with the role. I expect this role to mature and become more concrete as industry demand for it grows, rather than remaining the loosely defined, pipeline-focused title the book describes today.

The book's second core argument in this area, that a system-wide view is necessary because a model-centric focus misses how the model interacts with the rest of the system, holds directly in my own work. It shows, with evidence from Booking.com, that improvements in model accuracy do not always translate into improvements in system-level outcomes, and can occasionally hurt them. This is exactly why harness engineering matters in the systems I build: a more accurate model embedded in a poorly designed harness is not a better tool, and factors outside accuracy, latency, cost, and the reliability of the surrounding control flow, are often what determine whether the resulting system is actually usable in practice.

5 Paper Analysis

5.1 Vibe Coding and the Quality Assurance Gap

A recent grey literature review presented at ICSE SEIP 2026, *Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook* [6], studies "vibe coding": the practice of generating software through natural-language prompts and trial and error, without necessarily understanding the resulting code. The paper traces this practice to two main motivations, faster development and lowering the barrier to programming for non-developers, and describes a recurring psychological pattern behind it: an almost addictive sense of instant success, accompanied by frustration when it takes repeated refinements to get working code. Its central finding is what it calls a speed-quality paradox: users openly recognize that AI-generated code is often fast but flawed, yet they accept that risk to keep their development pace. This trade-off, the authors argue, produces a systemic breakdown in quality assurance, as testing is skipped, AI output is trusted uncritically, and bug fixing is delegated back to the same model that introduced the bug, ultimately creating a new class of "vulnerable developers" who can deploy applications, but have no clue of how to debug.

I found this claim to be consistent with my own observations. AI-generated code is fast but flawed, according to 68% of practitioners, while 19% describe it as fragile or error-prone. Although this is generally acknowledged, 36% of the behavior recorded in the review shows that users skip quality assurance completely, 18% trust the output without critical scrutiny, and 10% delegate bug-fixing directly to AI, leaving

29% who carry out manual testing or editing. There is a structural mismatch here rather than a discipline issue among developers: the speed of AI-assisted code generation has increased far faster than the speed at which that code can be tested, so teams face a real bottleneck between building software quickly and validating it at the same pace. Skipping validation to keep up produces exactly the kind of technical debt discussed earlier in this document; doing validation properly, with hand-written test scripts and manual technique selection, uses most of the time saved by using AI in the first place.

As a result, I argue AI-based systems should target this gap directly rather than simply focusing on code generation. If LLMs can write software faster than a team can verify it, the same capability should be directed at automating verification itself, using domain-specific reasoning and a modular system design that isolates and checks individual procedures rather than trusting a monolithic output, and keeping a human in the loop rather than delegating both the writing and the fixing of code to the same model. This is the motivation behind my recent work on agentic systems for Android GUI testing, and more broadly behind my interest in identifying gaps like this one in the software engineering process and designing AI frameworks that close them rather than widen them.

5.2 Evaluation of AI Agents and the SWE-Bench Problem

A recent paper at CAIN 2026, *Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation* [7], addresses a limitation in how AI software engineering agents are currently evaluated. Benchmarks such as SWE-bench test agents on detailed, well-formed GitHub issue reports, but developers interacting with coding agents in practice typically write short, informal, and sometimes incomplete messages. In this regard, the paper introduces the "mutation" framework that rewrites detailed benchmark problems into realistic, less polished user queries grounded in actual developer behavior. The authors first analyzed 10,000 real user queries to identify common templates for how developers report bugs and phrase requests to these systems, then used an LLM to rewrite existing formal SWE-bench problems into that informal, template-based style. Using the resulting dataset, they found that agent performance dropped significantly compared to the original benchmark. This led them to conclude that standard benchmarks substantially overestimate an agent's real-world bug-fixing capability.

This finding supports an argument I make earlier in this report: robustness to how a request is phrased is not a side concern but a core property that needs to be tested directly. Where my discussion of metamorphic testing focuses on paraphrases and dialect variation preserving a fixed intent, in this paper, the concern is extended to a different axis, informal, underspecified, and incomplete phrasing, and shows that current benchmarks miss it entirely because they only ever present a model with one clean version of a request. This reflects the same underlying principle as the metamorphic testing argument above. A system's input distribution at benchmark time and at deployment time are not the same, and a score obtained on one clean phrasing cannot detemine behavior once phrasing shifts to what users actually write.

The paper's finding also motivates the case for continued research on prompt optimization and reasoning. If an agent can recover a user's real intent from a messy, incomplete prompt and turn it into a precise internal instruction before acting, this gap should narrow. As prompt optimization and reasoning improve, LLMs should handle informal, real-world requests more reliably. This direction can make these tools usable by a wider range of people who cannot write precise issue explanations in the first place. Practically, the mutation framework itself is directly useful to my own work. It lets me test a prompt-optimization or reasoning framework against realistically degraded inputs, not just curated benchmark phrasing. That is an appropriate evaluation needed to check whether such a framework holds up in real-world software engineering settings, not just on SWE-bench as originally formatted.

6 Evidence, Originality, and GenAI-Use Transparency

- **Sources.** This essay references the papers, book chapters, and other sources in the bibliography, as well as the slides and material from the course lectures and guest lectures. It also includes relevant parts of my published research papers and licentiate thesis.
- **Source trust.** I trust peer-reviewed papers from top conferences and journals because they cover the latest research and undergo a review process that filters out weaker claims.
- **GenAI usage.** I used a GenAI tool to summarize an earlier, longer draft and to check for errors and improve the text, always confirming the changes myself. For proofreading, I asked the tool to highlight only unclear or poorly written sentences, pointing out grammatical mistakes. It provided the original sentence, a suggested revision, and an explanation for each change. This way, I could review the suggestions and choose only the improvements I agreed with, instead of accepting a complete rewrite.

References

[1] Arsham Gholamzadeh Khoee. *Towards Domain-Centric Artificial Intelligence: Bridging General Capabilities and Domain-Specific Context*. Licentiate thesis, Chalmers Tekniska Hogskola (Sweden), 2026.

[2] Milan De Koning, Ali Asgari, Pouria Derakhshanfar, and Annibale Panichella. A metamorphic testing approach to diagnosing memorization in LLM-based program repair. *arXiv preprint arXiv:2604.21579*, 2026.

[3] Juyeon Yoon, Robert Feldt, and Shin Yoo. Intent-driven mobile GUI testing with autonomous large language model agents. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 129–139. IEEE, 2024.

[4] Arsham Gholamzadeh Khoee, Shuai Wang, Robert Feldt, Dhasarathy Parthasarathy, and Yinan Yu. GateLens: A reasoning-enhanced LLM agent for automotive software release analytics. *Journal of Systems and Software*, page 112961, 2026.

[5] Christian Kästner. *Machine Learning in Production: From Models to Products*. MIT Press, 2025.

[6] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. Vibe coding in practice: Motivations, challenges, and a future outlook—a grey literature review. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*, pages 212–223, 2026.

[7] Spandan Garg, Benjamin Steenhoek, and Yufan Huang. Saving SWE-bench: A benchmark mutation approach for realistic agent evaluation. *arXiv preprint arXiv:2510.08996*, 2025.

[8] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pages 8479–8565, 2026.